

# PRD: Ping

## 1. Product Overview

### Product Name

Ping

### One-Line Description

Ping helps users find local offers by creating structured requests, then matching them with nearby small businesses that fit their budget, timing, distance, and preferences.

### Core Concept

Users do not scroll endlessly through local business listings. They create a structured “Ping” describing what they need, and the app matches them with relevant local business offers.

Example:

“I need lunch after school, under \$15, within 3 km, open between 3:30 PM and 5:00 PM.”

Ping returns ranked local business offers, explains why each offer matched, lets users claim/save offers, verifies redemption, then unlocks reviews and personal ranking.

### Competition Goal

Build a standalone application for the FBLA Coding & Programming event that demonstrates strong technical skill, clear UX, full prompt alignment, meaningful data analysis, intelligent matching, input validation, and polished presentation readiness.

---

## 2. Problem Statement

### User Problem

People often want to support local businesses but do not want to waste time comparing dozens of listings, ratings, menus, prices, and hours. Existing tools show what exists nearby, but they do not directly match a user’s specific need to local business offers.

## Business Problem

Small local businesses struggle to reach nearby customers at the exact moment those customers are ready to buy. They may have slow hours, unused appointments, expiring inventory, or student discounts that customers never discover.

## Product Opportunity

Ping creates a two-sided local discovery tool:

1. Users describe what they need.
  2. Local businesses publish structured offers.
  3. The app matches the two sides using a scoring algorithm.
  4. Claims, redemptions, reviews, bookmarks, and reports create measurable local support.
- 

## 3. Target Users

### Primary User: Local Customer

Example users:

- High school students
- Families
- Young adults
- Local residents
- People looking for nearby deals

Needs:

- Find relevant local businesses quickly
- Save money
- Discover offers
- Avoid unreliable reviews
- Save favorite businesses
- Track local spending/support

### Secondary User: Small Business Owner

Example businesses:

- Cafes
- Restaurants
- Barbers
- Salons

- Phone repair shops
- Tutoring centers
- Bookstores
- Gyms
- Thrift stores
- Entertainment businesses

Needs:

- Post offers
- Attract nearby customers
- Fill slow hours
- Track claims and redemptions
- See reviews and customer trends

## **Tertiary User: FBLA Judge**

Needs:

- Understand the product quickly
  - See that the app addresses the prompt
  - See working functionality
  - See intelligent features
  - See clean code structure
  - See data storage and reports
  - See clear UX rationale
- 

# **4. Product Positioning**

## **Positioning Statement**

Ping is a local offer-matching app that helps users discover and support small businesses by turning specific user needs into verified, claimable local offers.

## **What Makes Ping Different**

Most business discovery tools are built around browsing listings. Ping is built around structured demand.

The user does not start with:

“Show me all restaurants near me.”

The user starts with:

"I need lunch under \$15 after school."

The product then matches that need with real local offers.

## **Main User Benefit**

Ping saves users time and money by turning a specific need into a short ranked list of local offers they can claim immediately.

## **Main Business Benefit**

Ping helps small businesses reach customers with timely offers and gives them analytics on views, claims, redemptions, reviews, and customer interest.

---

# **5. Product Principles**

## **1. Structured, Not Chaotic**

Users should not be able to post absurd freeform requests like:

"I want a haircut for free within 15 minutes."

The request flow must use structured inputs, preset ranges, semantic validation, and category-specific rules.

## **2. Minimal Visual Design**

The app should look clean, simple, and modern. Avoid clutter. The interface should feel easy enough for a judge to understand within seconds.

## **3. Clear Rubric Alignment**

Every major feature should visibly support the FBLA topic or rating sheet:

- Categories
- Reviews/ratings
- Sorting
- Bookmarks
- Deals/coupons
- Bot prevention
- Intelligent feature
- Validation
- Reports

- Data storage
- UX and accessibility

## 4. Standalone and Demo-Safe

The app must run without internet. Use seeded local data. Do not depend on live APIs, real maps, payments, SMS, or external authentication.

## 5. Explainable Intelligence

The app should not just say “best match.” It should explain why an offer was ranked highly.

Example:

“This offer fits your budget, is 1.8 km away, is open after school, has a verified 4.7 rating, and includes a student deal.”

---

## 6. Success Metrics

### Product Metrics

- User can create a valid Ping request in under 60 seconds.
- User can claim an offer in under 3 clicks after viewing matches.
- Business can create an offer in under 90 seconds.
- Reports update after claims, redemptions, and reviews.
- App can run offline with seeded data.

### Competition Metrics

The app should demonstrate:

- Complete prompt coverage
  - Strong UX design rationale
  - Intelligent matching
  - Semantic and syntactic input validation
  - Modular code
  - Data structures
  - Customizable reports
  - Clear demo story
  - No runtime errors
-

## 7. MVP Scope

### Must-Have Features

1. User login/mock profile selector
2. Home dashboard
3. Structured Ping request builder
4. Semantic input validation
5. OfferRank intelligent matching engine
6. Matched offers results page
7. Business profiles
8. Category browsing and sorting
9. Save/bookmark businesses
10. Save/bookmark offers
11. Claim offer flow
12. Claim code generation
13. Business redemption flow
14. Verified review/rating flow
15. Pairwise comparison ranking
16. User report dashboard
17. Business owner dashboard
18. Business offer creation
19. Business analytics report
20. Help panel/tooltips
21. Local data persistence
22. Reset demo data button

### Nice-to-Have Features

1. Friend preference signal using seeded friends
2. Group request mode
3. Accessibility preference toggle
4. Dark mode
5. Export report as mock CSV
6. Offer expiration countdown
7. Business verification badge

### Out of Scope

Do not build:

- Real payments
- Real SMS verification
- Real Google Maps integration
- Real-time chat
- Live business onboarding

- Live AI chatbot
  - Actual geolocation dependency
  - Actual email sending
  - Real QR code scanning
  - Public deployment dependency for demo
- 

## 8. Core User Stories

### User Request Creation

As a user, I want to describe what I need locally using simple structured fields so that the app can match me with relevant businesses.

Acceptance criteria:

- User can select category.
- User can select need type.
- User can choose budget.
- User can choose distance.
- User can choose time window.
- User can add optional preferences.
- User can add optional note.
- App blocks unrealistic requests.
- App explains validation errors clearly.

### Offer Matching

As a user, I want to see a ranked list of local offers that fit my request so that I can choose quickly.

Acceptance criteria:

- App returns top matching offers.
- Each offer has a match score.
- Each offer explains why it matched.
- User can sort results.
- User can filter results.
- User can save or claim an offer.

### Claiming Offers

As a user, I want to claim an offer and receive a verification code so that the business can confirm my redemption.

Acceptance criteria:

- Claim code is generated.
- Claim status is stored.
- User cannot claim expired or full offers.
- User cannot claim the same offer twice.
- Active claims appear in user dashboard.

## Business Redemption

As a business owner, I want to enter a user's claim code so that I can mark the offer as redeemed.

Acceptance criteria:

- Business can search claim code.
- Business can mark valid claim as redeemed.
- Invalid or expired codes are rejected.
- Redemption updates analytics.

## Reviews

As a user, I want to leave a verified review after redeeming an offer so that other users can trust the rating.

Acceptance criteria:

- Reviews unlock only after redemption.
- Rating must be 1 to 5.
- Review text has a minimum and maximum length.
- Review is attached to business and claim.
- Duplicate reviews for same claim are blocked.

## Bookmarks

As a user, I want to save businesses and offers so that I can return to them later.

Acceptance criteria:

- User can save business.
- User can save offer.
- User can remove saved item.
- Saved page can be sorted by category, rating, or deal expiration.

## Reports

As a user, I want a report showing how many local businesses I supported and how much I saved.



Acceptance criteria:

- Report shows total offers claimed.
- Report shows redeemed offers.
- Report shows estimated savings.
- Report shows businesses supported.
- Report shows favorite categories.
- Report has date/category filters.

## Business Analytics

As a business owner, I want a dashboard showing offer performance so that I can understand customer interest.

Acceptance criteria:

- Dashboard shows offer views.
  - Dashboard shows claims.
  - Dashboard shows redemptions.
  - Dashboard shows conversion rate.
  - Dashboard shows average rating.
  - Dashboard shows common review tags.
  - Dashboard supports filtering by offer/date.
- 

# 9. Information Architecture

## Main User Navigation

Use a left sidebar on desktop and bottom tab bar on mobile.

Pages: 1. Home 2. Create Ping 3. Matches 4. Explore 5. Saved 6. Claims 7. Rankings 8. Reports 9. Help

## Business Navigation

Pages: 1. Business Dashboard 2. Create Offer 3. Offers 4. Redeem Claim 5. Reviews 6. Analytics 7. Business Profile

## Admin/Demo Navigation

Pages: 1. Demo Controls 2. Reset Data 3. Seed Data Viewer

---

# 10. Full User UI Specification

## 10.1 Home Page

### Purpose

Give the user an immediate action: create a Ping or browse active offers.

### Layout

Top section:

- Greeting
- Location context
- Main CTA: "Create a Ping"
- Secondary CTA: "Browse Offers"

Cards:

- Active claims
- Saved businesses
- Recent offers
- Top categories
- Money saved this month

### Example Copy

Header:

What do you need nearby?

Subtext:

Match with local businesses based on your budget, timing, distance, and preferences.

Main button:

Create a Ping

Secondary button:

Explore Businesses

---

## 10.2 Create Ping Page

### Purpose

Let users create a structured local request.

### Design Style

Minimal visual design with a guided sentence builder.

### Main Layout

Two-column desktop layout.

Left column:

- Request builder inputs

Right column:

- Live preview card
- Estimated matches
- Request quality
- Validation messages

Mobile:

- Single column
- Preview card becomes sticky bottom summary

### Sentence Builder

Display:

I need [category] for [need] within [distance] around [time] for [budget].

Each bracket is an interactive control.

### Field 1: Category

Use large selectable cards.

Options:

- Food
- Retail
- Services

- Fitness
- Education
- Repair
- Entertainment

Each category has:

- Icon
- Label
- Short description

## **Field 2: Need Type**

Need type options depend on category.

Food:

- Lunch
- Cafe/study spot
- Dessert
- Dinner
- Group meal
- Quick snack

Retail:

- Gift
- Clothing
- Books
- Thrift item
- School supplies
- Home item

Services:

- Haircut
- Salon service
- Printing
- Alterations
- Tutoring
- Cleaning

Fitness:

- Gym trial
- Drop-in class
- Sports facility
- Personal training

Education:

- Tutoring
- Test prep
- Workshop
- Study space

Repair:

- Phone repair
- Laptop repair
- Bike repair
- Clothing repair

Entertainment:

- Escape room
- Arcade
- Movie/activity
- Local event
- Group hangout

### **Field 3: Budget**

Use preset chips based on need type.

Food:

- Under \$10
- Under \$15
- Under \$20
- No budget

Haircut:

- \$20 to \$30
- \$30 to \$45
- \$45+

Phone repair:

- Under \$50
- \$50 to \$100
- \$100+

Gift:

- Under \$15
- Under \$25

- Under \$50

Also include:

- Custom budget

#### **Field 4: Distance**

Options:

- Within 1 km
- Within 3 km
- Within 5 km
- Within 10 km

Display current demo origin:

- White Oaks Secondary School
- Downtown Oakville
- User profile location

Use seeded coordinates.

#### **Field 5: Time Window**

Options:

- Now
- After school
- Tonight
- Tomorrow
- This weekend
- Custom

Custom:

- Date
- Start time
- End time

Validation:

- End time must be after start time.
- Time cannot be in the past.
- Time window cannot exceed 7 days for a request.

#### **Field 6: Preferences**

Use toggle chips.

Options:

- Student discount
- Open now
- Highly rated only
- Verified businesses only
- Group-friendly
- Wheelchair accessible
- Quiet environment
- Vegetarian options
- Fast service
- Under 30 minutes

Only show relevant preferences by category.

### Field 7: Optional Note

Text area:

- Placeholder: "Example: need outlets and a quiet table"
- Max length: 120 characters
- No links
- No repeated spam text
- No profanity list for demo
- Optional

### Live Preview Card

Example:

Title:

Your Ping

Body:

Lunch after school  
Under \$15  
Within 3 km  
Open between 3:30 PM and 5:00 PM  
Student deals preferred

Footer:

Estimated matches: 8  
Request quality: Strong

## Request Quality Rules

Strong:

- Category selected
- Need selected
- Budget valid
- Distance selected
- Time selected

Weak:

- Missing budget
- Missing time
- Very broad distance
- No preferences

Invalid:

- Unrealistic budget
- Invalid time
- Missing required fields
- Duplicate request within cooldown

## Primary CTA

Button:

Find Matching Offers

Disabled until all required fields are valid.

## Validation Error Examples

Budget too low:

Your budget is below the typical minimum for this request type. Try a higher range or browse existing deals.

Time invalid:

End time must be after start time.

Duplicate request:

You already created a similar Ping recently. Try editing that request or wait before creating another.



---

## 10.3 Verification Step

### Purpose

Prevent bot-like activity and satisfy prompt requirement.

### Display After User Clicks “Find Matching Offers”

Modal title:

Quick verification

Content:

- Checkbox: “I agree to only claim offers I intend to use.”
- Mock verification code: user must type “2468”
- Submit button

### Rules

- User must check box.
- User must enter correct mock code.
- User has max 3 attempts.
- Failed attempts show helpful errors.
- Verification status is stored in session.

---

## 10.4 Matches Page

### Purpose

Show ranked matched offers.

### Layout

Top:

- Request summary
- Number of matching offers
- Sorting dropdown
- Filter chips

Sorting options:

- Best match

- Highest rating
- Closest
- Lowest price
- Ending soon
- Most claimed

Filters:

- Active deals
- Open now
- Student discount
- Verified reviews
- Saved businesses

## Offer Card

Each card must show:

- Business name
- Business category
- Offer title
- Offer price
- Original price if applicable
- Savings
- Distance
- Rating
- Review count
- Time availability
- Match score
- Why it matched
- Claim button
- Save button
- View business button

Example:

FreshBowl Cafe  
 Student lunch bowl: \$11.99  
 1.8 km away | 4.7 rating | 42 reviews  
 Match: 91%

Why this matched:

- Fits your \$15 budget
- Open after school
- Has student discount
- Highly rated by students

Buttons:

- Claim Offer
- Save
- Details

## Empty State

If no matches: Title:

No exact matches found

Suggestions:

- Increase distance
- Raise budget
- Change time window
- Browse similar offers

Show 3 near-miss offers with reasons.

---

## 10.5 Claim Offer Flow

### Purpose

Allow user to claim a valid offer.

### Flow

1. User clicks "Claim Offer."
2. App checks if offer is still available.
3. App checks if user already claimed it.
4. App checks if user claim limit is reached.
5. App generates claim code.
6. App stores claim.
7. App shows confirmation.

### Claim Confirmation Screen

Show:

- Claim code
- Business name
- Offer title
- Expiration time
- Redemption instructions

- Button to add to active claims
- Button to view business

Example:

Claim code: PING-8421

Show this code at FreshBowl Cafe before 5:00 PM.

## Claim Statuses

- Active
- Redeemed
- Expired
- Cancelled

## Claim Rules

- Cannot claim expired offer.
  - Cannot claim if max claims reached.
  - Cannot claim same offer twice.
  - Cannot have more than 3 active claims.
  - Claim expires based on offer validity.
- 

# 10.6 Claims Page

## Purpose

Show user's active and past claims.

Tabs:

- Active
- Redeemed
- Expired

Active claim card:

- Business
- Offer
- Code
- Expires in
- Directions mock button
- Cancel claim button

Redeemed card:

- Business

- Offer
- Redeemed date
- Leave review button if no review exists

Expired card:

- Business
  - Offer
  - Expired date
  - Find similar offers button
- 

## 10.7 Review Flow

### Purpose

Allow verified reviews after redemption.

### Unlock Rule

Review button appears only after claim status is redeemed.

### Review Form

Fields:

- Rating from 1 to 5
- Review text
- Tags
- Would recommend toggle
- Better/worse comparison prompt

Tags:

- Good value
- Friendly staff
- Fast service
- Student-friendly
- Clean
- Good quality
- Quiet
- Group-friendly

Validation:

- Rating required.
- Rating must be 1 to 5.

- Text minimum 10 characters.
- Text maximum 300 characters.
- User cannot review same claim twice.
- Review must be tied to redeemed claim.

## **Review Display**

Business reviews should show:

- Rating
  - Text
  - Tags
  - Verified badge
  - Date
  - User first name or mock username
- 

## **10.8 Pairwise Comparison Ranking**

### **Purpose**

Build personal rankings from user comparisons.

### **Trigger**

After user submits a review, show:

Help us place this business in your personal rankings.

Example:

Was FreshBowl Cafe better value than Bowl & Co?

Buttons:

- Better
- Worse
- About the same
- Skip

### **Algorithm**

Use binary insertion into the user's ranked list for that category.

Example:

- User has 8 ranked lunch spots.

- App compares new business to middle-ranked business.
- Based on answer, app searches upper or lower half.
- Repeat until correct insertion index is found.

## UX

Keep it quick. Show max 3 comparisons during demo unless the list is larger.

### Ranking Page

Show ranked lists by category:

- Best lunch spots
- Best cafes
- Best haircut places
- Best study spots
- Best value deals

Each ranked card:

- Rank number
  - Business name
  - Category
  - Rating
  - Last visit
  - Save status
- 

## 10.9 Explore Page

### Purpose

Clearly satisfy prompt features around business discovery, categories, sorting, ratings, and bookmarks.

### Layout

Search/filter header:

- Search businesses
- Category dropdown
- Rating filter
- Distance filter
- Deal availability filter
- Sort dropdown

Business grid:

- Cards with business details

### Sort Options

- Category
- Highest rating
- Most reviews
- Closest
- Most saved
- Active deals
- Alphabetical

### Business Card

Show:

- Business name
  - Category
  - Rating
  - Review count
  - Distance
  - Active deal count
  - Verified badge
  - Save button
  - View profile button
- 

## 10.10 Business Profile Page

### Purpose

Show details for each business.

Sections: 1. Header 2. Rating summary 3. Active offers 4. Reviews 5. Business details 6. Similar businesses

Header:

- Business name
- Category
- Rating
- Review count
- Distance
- Save button
- Verified badge



Details:

- Address
- Hours
- Price range
- Tags
- Accessibility info
- Student discount status

Active offers:

- Offer cards
- Claim button
- Expiration info

Reviews:

- Verified reviews first
  - Sort by newest, highest, lowest
  - Filter by tags
- 

## 10.11 Saved Page

### Purpose

Show saved businesses and offers.

Tabs:

- Businesses
- Offers

Saved businesses can be sorted by:

- Category
- Rating
- Distance
- Recently saved
- Active deal count

Saved offers can be sorted by:

- Ending soon
- Highest savings
- Business rating
- Category

Empty state:

You have not saved anything yet. Explore local businesses or create a Ping to find offers.

---

## 10.12 User Reports Page

### Purpose

Show customizable user data analysis.

Filters:

- Date range
- Category
- Claim status
- Business type

Metrics:

- Total offers claimed
- Offers redeemed
- Estimated money saved
- Businesses supported
- Reviews submitted
- Favorite category
- Average rating given
- Top ranked businesses

Charts:

- Claims by category
- Savings over time
- Businesses supported by month
- Rating distribution

Report cards:

- "You supported 6 local businesses this month."
- "You saved an estimated \$42."
- "Your top category is Food."

Data must update when user claims, redeems, reviews, or saves.

---

# 11. Business UI Specification

## 11.1 Business Dashboard

### Purpose

Give business owner overview of performance.

Metrics:

- Active offers
- Total views
- Claims
- Redemptions
- Conversion rate
- Average rating
- Review count
- Most popular offer

Charts:

- Claims over time
- Redemptions by offer
- Rating trend
- Customer tags

CTA:

- Create Offer
  - Redeem Claim
- 

## 11.2 Create Offer Page

### Purpose

Let businesses create structured offers.

Fields:

- Offer title
- Category
- Offer type
- Description
- Price
- Original price

- Start date/time
- End date/time
- Max claims
- Tags
- Student discount toggle
- Verification required toggle

Validation:

- Title required.
- Category required.
- Price must be non-negative.
- Original price must be greater than offer price if entered.
- End time must be after start time.
- Max claims must be at least 1.
- Offer title maximum 80 characters.
- Description maximum 250 characters.

### Offer Types

- Discount
- Limited-time deal
- Student offer
- Group offer
- Appointment slot
- Event
- Free trial
- Bundle

---

## 11.3 Offers Management Page

Tabs:

- Active
- Scheduled
- Expired

Offer card:

- Title
- Price
- Dates
- Claims
- Redemptions
- Views
- Status

- Edit button
  - Deactivate button
- 

## 11.4 Redeem Claim Page

### Purpose

Allow business to verify a user claim code.

Input:

- Claim code text field

Button:

- Verify Claim

Validation:

- Code format must match PING-####.
- Claim must exist.
- Claim must belong to business offer.
- Claim must be active.
- Claim must not be expired.

Success state:

Claim verified. Mark as redeemed?

After redeeming:

Claim redeemed successfully. User can now leave a verified review.

Failure states:

- Invalid code
  - Expired claim
  - Already redeemed
  - Claim belongs to another business
-

## 11.5 Business Reviews Page

Show:

- Average rating
- Review count
- Verified review percentage
- Review list
- Tag summary

Filters:

- Rating
  - Date
  - Offer
  - Tag
- 

## 11.6 Business Analytics Page

Metrics:

- Offer views
- Claims
- Redemptions
- Conversion rate
- Average rating
- Repeat customers
- Most common review tags
- Revenue influenced estimate

Formula:  $\text{Conversion rate} = \text{redeemed claims} / \text{offer views}$

$\text{Revenue influenced estimate} = \text{redeemed claims times offer price}$

Display formulas in UI tooltips.

---

## 12. Data Model

Use TypeScript types.

## User

```
export type UserRole = "customer" | "businessOwner" | "admin";

export type User = {
  id: string;
  name: string;
  email: string;
  role: UserRole;
  homeLocationId: string;
  verified: boolean;
  createdAt: string;
  preferences: UserPreferences;
};
```

## UserPreferences

```
export type UserPreferences = {
  preferredCategories: BusinessCategory[];
  maxDefaultDistanceKm: number;
  studentDiscountPreferred: boolean;
  accessibilityNeeds: string[];
  savedBusinessIds: string[];
  savedOfferIds: string[];
};
```

## Business

```
export type BusinessCategory =
  | "food"
  | "retail"
  | "services"
  | "fitness"
  | "education"
  | "repair"
  | "entertainment";

export type Business = {
  id: string;
  name: string;
  category: BusinessCategory;
  description: string;
  address: string;
```

```
location: GeoPoint;
hours: BusinessHours[];
ratingAverage: number;
reviewCount: number;
verified: boolean;
priceLevel: 1 | 2 | 3 | 4;
tags: string[];
accessibilityFeatures: string[];
ownerUserId: string;
createdAt: string;
};
```

## GeoPoint

```
export type GeoPoint = {
  lat: number;
  lng: number;
};
```

## BusinessHours

```
export type BusinessHours = {
  dayOfWeek: number;
  openTime: string;
  closeTime: string;
};
```

## Offer

```
export type OfferType =
  | "discount"
  | "limitedTime"
  | "studentOffer"
  | "groupOffer"
  | "appointmentSlot"
  | "event"
  | "freeTrial"
  | "bundle";

export type Offer = {
  id: string;
  businessId: string;
  title: string;
```



```
description: string;
category: BusinessCategory;
offerType: OfferType;
price: number;
originalPrice?: number;
validFrom: string;
validUntil: string;
maxClaims: number;
currentClaims: number;
views: number;
tags: string[];
studentOnly: boolean;
verificationRequired: boolean;
active: boolean;
createdAt: string;
};
```

## PingRequest

```
export type NeedType =
  | "lunch"
  | "cafeStudySpot"
  | "dessert"
  | "dinner"
  | "groupMeal"
  | "gift"
  | "clothing"
  | "books"
  | "thrift"
  | "schoolSupplies"
  | "haircut"
  | "printing"
  | "alterations"
  | "tutoring"
  | "gymTrial"
  | "phoneRepair"
  | "laptopRepair"
  | "escapeRoom"
  | "localEvent"
  | "groupHangout";

export type PingRequest = {
  id: string;
  userId: string;
  category: BusinessCategory;
  needType: NeedType;
```

```

    budgetMin?: number;
    budgetMax?: number;
    distanceKm: number;
    timeStart: string;
    timeEnd: string;
    preferences: string[];
    optionalNote?: string;
    verifiedHuman: boolean;
    status: "draft" | "submitted" | "matched" | "expired";
    createdAt: string;
};

```

## MatchResult

```

export type MatchResult = {
  offerId: string;
  businessId: string;
  requestId: string;
  score: number;
  scoreBreakdown: ScoreBreakdown;
  reasons: string[];
};

```

## ScoreBreakdown

```

export type ScoreBreakdown = {
  categoryScore: number;
  budgetScore: number;
  distanceScore: number;
  ratingScore: number;
  timeScore: number;
  verificationScore: number;
  preferenceScore: number;
};

```

## Claim

```

export type ClaimStatus = "active" | "redeemed" | "expired" | "cancelled";

export type Claim = {
  id: string;
  userId: string;
  offerId: string;
};

```

```
businessId: string;
claimCode: string;
status: ClaimStatus;
createdAt: string;
expiresAt: string;
redeemedAt?: string;
};
```

## Review

```
export type Review = {
  id: string;
  userId: string;
  businessId: string;
  offerId: string;
  claimId: string;
  rating: number;
  text: string;
  tags: string[];
  verified: boolean;
  createdAt: string;
};
```

## SavedBusiness

```
export type SavedBusiness = {
  userId: string;
  businessId: string;
  savedAt: string;
  tags: string[];
};
```

## SavedOffer

```
export type SavedOffer = {
  userId: string;
  offerId: string;
  savedAt: string;
};
```

## PersonalRanking

```
export type PersonalRanking = {  
  userId: string;  
  category: BusinessCategory;  
  needType?: NeedType;  
  rankedBusinessIds: string[];  
  updatedAt: string;  
};
```

---

## 13. Algorithms and Logic

### 13.1 OfferRank Matching Algorithm

#### Purpose

Rank offers by how well they fit a Ping request.

#### Formula

Score =

0.25 times category score

- 0.20 times budget score
- 0.15 times distance score
- 0.15 times rating score
- 0.10 times time score
- 0.10 times verification score
- 0.05 times preference score

Each subscore is from 0 to 100.

#### Score Fields

Category score:

- 100 if offer category matches request category
- 60 if related category
- 0 if unrelated

Budget score:

- 100 if offer price is within budget

- 70 if offer price is within 15 percent above budget
- 30 if offer price is above budget but has strong rating
- 0 if far above budget

Distance score:

- 100 if business is within selected distance
- Scales down as distance approaches max
- 0 if outside max distance

Rating score:

- Convert average rating from 1 to 5 into 0 to 100
- Example: 4.5 rating becomes 90

Time score:

- 100 if offer and business are available during request window
- 50 if partially available
- 0 if unavailable

Verification score:

- 100 if business is verified and offer requires redemption
- 70 if business is verified
- 40 if unverified

Preference score:

- Add points for matching preferences like student discount, open now, group-friendly, accessible, quiet, vegetarian, etc.

## Match Reasons

Generate human-readable reasons from score components.

Example:

- "Fits your budget"
- "Open during your requested time"
- "Within 3 km"
- "Has a student discount"
- "Highly rated by verified reviews"

## 13.2 Semantic Validation

### Purpose

Prevent absurd or low-quality requests.

### Validation Layers

Syntactic validation:

- Required fields present
- Budget is numeric
- Distance is numeric
- Time is valid
- Note length is valid

Semantic validation:

- Budget must be realistic for need type
- Time window must make sense
- Distance cannot exceed maximum
- User cannot spam duplicate requests
- User cannot create too many active requests
- Optional note cannot contain links or repeated nonsense

### Category Minimum Budgets

```
export const minimumBudgetByNeedType: Record<NeedType, number> = {  
  lunch: 8,  
  cafeStudySpot: 5,  
  dessert: 5,  
  dinner: 12,  
  groupMeal: 15,  
  gift: 10,  
  clothing: 10,  
  books: 5,  
  thrift: 5,  
  schoolSupplies: 3,  
  haircut: 20,  
  printing: 2,  
  alterations: 8,  
  tutoring: 15,  
  gymTrial: 0,  
  phoneRepair: 40,  
  laptopRepair: 50,  
  escapeRoom: 20,  
  localEvent: 0,
```

```
groupHangout: 10
};
```

### Example Error

If needType is haircut and budgetMax is 5:

Haircuts usually require a higher budget. Try \$20 to \$30 or browse existing deals.

---

## 13.3 Claim Code Generation

Format:

- PING-####

Example:

- PING-8421

Rules:

- Must be unique among active claims.
  - Must expire with the offer.
  - Must be tied to user, offer, and business.
  - Must be redeemable only once.
- 

## 13.4 Review Verification

A review is verified only if:

- User has a claim.
  - Claim is redeemed.
  - Review references that claim.
  - No previous review exists for same claim.
- 

## 13.5 Pairwise Ranking Algorithm

### Purpose

Rank businesses based on user comparison, rather than only 1 to 5 ratings.

## Trigger

After review submission.

## Logic

Use binary insertion.

Input:

- Existing ranked list
- New business
- Comparison answers

Process: 1. Compare new business to middle item. 2. If user says new business is better, search upper half. 3. If worse, search lower half. 4. Repeat until insertion point is found. 5. Insert new business into ranked list.

## Complexity

Linear insertion may require up to  $n$  comparisons.

Binary insertion requires about  $\log_2 n$  comparisons.

Example: If the user has 64 ranked businesses, binary insertion takes about 6 comparisons.

## UX Rule

Limit demo comparison to 3 prompts unless user chooses to continue.

---

## 13.6 Report Calculations

User estimated savings:

- Sum of originalPrice minus price for redeemed offers where originalPrice exists.

Business conversion rate:

- Redeemed claims divided by offer views.

Claim redemption rate:

- Redeemed claims divided by total claims.

Average rating:

- Sum of ratings divided by review count.



---

## 14. Technical Requirements

### Recommended Stack

- React
- TypeScript
- Vite
- LocalStorage or IndexedDB
- Recharts or Chart.js
- CSS Modules or Tailwind
- No required backend for MVP
- No required internet connection

### App Type

Standalone interactive web app.

### Offline Requirement

The app must work with seeded local data:

- Seed users
- Seed businesses
- Seed offers
- Seed claims
- Seed reviews
- Seed rankings

### Data Persistence

Use localStorage for simpler implementation or IndexedDB for cleaner structured storage.

Minimum:

- Persist created requests
- Persist claims
- Persist saved businesses
- Persist reviews
- Persist redemptions
- Persist reports after refresh

### Demo Reset

Add a "Reset Demo Data" button under Admin/Demo Controls.

---

## 15. Recommended File Structure

```
src/  
  app/  
    App.tsx  
    routes.tsx  
    providers.tsx  
  
  models/  
    User.ts  
    Business.ts  
    Offer.ts  
    PingRequest.ts  
    Claim.ts  
    Review.ts  
    Ranking.ts  
    Report.ts  
  
  data/  
    seedUsers.ts  
    seedBusinesses.ts  
    seedOffers.ts  
    seedClaims.ts  
    seedReviews.ts  
    seedRankings.ts  
  
  services/  
    storageService.ts  
    requestValidationService.ts  
    offerMatchingService.ts  
    claimService.ts  
    reviewService.ts  
    rankingService.ts  
    reportService.ts  
    businessService.ts  
    userService.ts  
  
  utils/  
    distance.ts  
    dateTime.ts  
    formatting.ts  
    sorting.ts  
    validation.ts  
    ids.ts
```

```
components/  
  layout/  
    Sidebar.tsx  
    TopBar.tsx  
    MobileNav.tsx  
    PageHeader.tsx  
  
  common/  
    Button.tsx  
    Card.tsx  
    Badge.tsx  
    Modal.tsx  
    EmptyState.tsx  
    HelpTooltip.tsx  
    FormError.tsx  
  
  ping/  
    CategorySelector.tsx  
    NeedTypeSelector.tsx  
    BudgetSelector.tsx  
    DistanceSelector.tsx  
    TimeWindowSelector.tsx  
    PreferenceChips.tsx  
    RequestPreview.tsx  
    VerificationModal.tsx  
  
  offers/  
    OfferCard.tsx  
    MatchScoreBadge.tsx  
    MatchReasons.tsx  
    ClaimCodeCard.tsx  
  
  businesses/  
    BusinessCard.tsx  
    BusinessProfileHeader.tsx  
    BusinessReviews.tsx  
    ActiveOffers.tsx  
  
  reports/  
    MetricCard.tsx  
    ClaimsByCategoryChart.tsx  
    SavingsChart.tsx  
    RatingDistributionChart.tsx  
    ReportFilters.tsx  
  
pages/  
  HomePage.tsx  
  CreatePingPage.tsx
```

```
MatchesPage.tsx
ExplorePage.tsx
BusinessProfilePage.tsx
SavedPage.tsx
ClaimsPage.tsx
RankingsPage.tsx
UserReportsPage.tsx
BusinessDashboardPage.tsx
CreateOfferPage.tsx
ManageOffersPage.tsx
RedeemClaimPage.tsx
BusinessAnalyticsPage.tsx
HelpPage.tsx
DemoControlsPage.tsx

styles/
  globals.css
  tokens.css
```

---

## 16. Service Function Requirements

### **requestValidationService.ts**

Functions:

- validatePingRequest(request)
- validateBudgetForNeedType(needType, budgetMax)
- validateTimeWindow(start, end)
- validateOptionalNote(note)
- detectDuplicateRequest(userId, request)
- getRequestQuality(request)

### **offerMatchingService.ts**

Functions:

- getMatchingOffers(request, offers, businesses, user)
- calculateOfferScore(request, offer, business, user)
- calculateCategoryScore(request, offer)
- calculateBudgetScore(request, offer)
- calculateDistanceScore(request, business)
- calculateRatingScore(business)
- calculateTimeScore(request, offer, business)
- calculateVerificationScore(business, offer)

- calculatePreferenceScore(request, offer, business, user)
- generateMatchReasons(scoreBreakdown, request, offer, business)

## **claimService.ts**

Functions:

- createClaim(userId, offerId)
- generateClaimCode()
- validateClaimCode(code)
- redeemClaim(code, businessId)
- expireOldClaims()
- getUserClaims(userId)
- getBusinessClaims(businessId)

## **reviewService.ts**

Functions:

- canUserReviewClaim(userId, claimId)
- createReview(reviewInput)
- validateReview(reviewInput)
- getBusinessReviews(businessId)
- updateBusinessRating(businessId)

## **rankingService.ts**

Functions:

- getRanking(userId, category, needType)
- startBinaryInsertion(userId, newBusinessId, category, needType)
- processComparison(answer)
- insertBusinessAtIndex(ranking, businessId, index)

## **reportService.ts**

Functions:

- getUserReport(userId, filters)
- getBusinessReport(businessId, filters)
- calculateEstimatedSavings(claims, offers)
- calculateConversionRate(views, redemptions)
- groupClaimsByCategory(claims, businesses)
- groupSavingsByMonth(claims, offers)

## storageService.ts

Functions:

- loadData()
  - saveData(data)
  - resetDemoData()
  - getCollection(collectionName)
  - updateCollection(collectionName, newData)
- 

# 17. Seed Data Requirements

## Users

Create at least:

- 3 customer users
- 2 business owner users
- 1 admin/demo user

Example:

- Lucas Chen, customer
- Maya Patel, customer
- Ethan Wong, customer
- Sam Rivera, business owner
- Nina Brooks, business owner

## Businesses

Create at least 20 businesses across categories.

Minimum by category:

- Food: 5
- Retail: 3
- Services: 4
- Fitness: 2
- Education: 2
- Repair: 2
- Entertainment: 2

Each business needs:

- Name

- Category
- Description
- Location
- Hours
- Rating
- Review count
- Tags
- Verified status
- Price level

## Offers

Create at least 30 offers.

Examples:

- Student lunch bowl, \$11.99
- \$3 iced latte after school
- \$25 student haircut
- 20 percent off phone screen protector
- Free gym trial class
- \$10 tutoring intro session
- Group escape room discount
- Blind date with a book, \$7
- Thrift hoodie drop
- Printing discount for students

## Reviews

Create at least 40 reviews.

Include:

- Ratings from 3 to 5
- Verified true/false
- Tags
- Different categories

## Claims

Create at least:

- 5 active claims
- 5 redeemed claims
- 3 expired claims

## Rankings

Create rankings for:

- Lunch
  - Cafes
  - Haircuts
  - Study spots
  - Gift shops
- 

# 18. UX Design System

## Visual Style

Clean, minimal, student-friendly, polished.

## Colors

Use a simple palette:

- Primary: deep blue or purple
- Background: off-white
- Cards: white
- Text: dark gray
- Success: green
- Warning: orange
- Error: red
- Neutral badges: light gray

## Typography

Use one clean sans-serif font.

Hierarchy:

- Page title: large and bold
- Section title: medium and bold
- Body: readable
- Helper text: smaller, muted



## Buttons

Primary:

- Filled background
- Clear label

Secondary:

- Outline or light background

Danger:

- Red text or red outline

Disabled:

- Lower opacity
- Cursor disabled

## Cards

Use cards for:

- Offers
- Businesses
- Claims
- Reports
- Saved items

Each card should have:

- Clear title
- Main metric or offer
- Supporting metadata
- Primary action
- Secondary action

## Accessibility

Include:

- Keyboard navigable controls
- Visible focus states
- Text labels, not icons alone
- Color contrast
- Form errors connected to fields

- Alt text for icons/images
  - Reduced motion option if animations exist
  - Accessible names for buttons
- 

## 19. Error and Empty States

### Invalid Request

This Ping needs a valid budget, distance, and time window before we can match offers.

### Low Budget

Your budget is below the typical minimum for this request type. Try a higher range or browse existing deals.

### No Matches

No exact matches found. Try increasing your distance, raising your budget, or changing your time window.

### Offer Full

This offer has reached its claim limit.

### Offer Expired

This offer has expired. View similar active offers.

### Invalid Claim Code

This claim code does not exist or is not active.

### Review Not Allowed

Reviews unlock after the offer is redeemed.

### Saved Empty

You have not saved any businesses or offers yet.

## Report Empty

Your report will appear after you claim or redeem offers.

---

## 20. Demo Script Requirements

The demo must be possible in 7 minutes.

### Demo Flow

1. Introduce Ping and problem.
2. Show home page.
3. Create a Ping for lunch under \$15 within 3 km after school.
4. Show structured inputs and live preview.
5. Try invalid budget for haircut to demonstrate semantic validation.
6. Submit valid Ping.
7. Complete mock verification.
8. Show OfferRank results.
9. Explain match score and reasons.
10. Claim an offer.
11. Show claim code.
12. Switch to business dashboard.
13. Redeem claim code.
14. Switch back to user.
15. Leave verified review.
16. Complete pairwise comparison ranking.
17. Show user report.
18. Show business analytics.
19. Briefly explain code architecture.

### Must Mention During Demo

- Runs standalone with local seeded data
  - Uses structured data types
  - Uses semantic validation
  - Uses OfferRank scoring algorithm
  - Uses binary insertion for rankings
  - Supports reviews, ratings, sorting, bookmarks, deals, and verification
  - Reports are customizable by date/category
-

## 21. Acceptance Criteria by Rubric Category

### Code Quality

Acceptance criteria:

- TypeScript types are defined for major entities.
- Code is split into models, services, components, pages, and utils.
- Matching logic is in service files, not inside UI components.
- Validation logic is reusable.
- Comments explain non-obvious logic.
- Naming is clear and consistent.
- No console errors during demo.

### UX Design

Acceptance criteria:

- User journey is easy to explain.
- Create Ping flow has clear steps.
- Interface has consistent layout.
- Help tooltips exist.
- Accessibility features are present.
- Validation errors are helpful.
- Navigation has no broken links.

### Functionality

Acceptance criteria:

- App addresses all topic requirements.
- Users can browse categories.
- Users can sort businesses by rating.
- Users can leave verified reviews.
- Users can save/bookmark businesses.
- Users can view and claim deals/coupons.
- App has bot-prevention verification.
- App has intelligent matching.
- App produces reports.
- App stores data locally.

## Data Storage

Acceptance criteria:

- Businesses, offers, claims, reviews, requests, saves, and rankings are stored.
- Data updates after user actions.
- Data persists after refresh.
- Variables and objects have clear roles.
- Arrays/lists are used appropriately.

## Presentation Readiness

Acceptance criteria:

- Demo data is ready.
  - Reset button works.
  - No internet is required.
  - All team members know their section.
  - Q&A answers are prepared.
  - Code can be shown if judges ask.
- 

## 22. Q&A Prep

### Why did you choose this idea?

Because the prompt asks us to help users discover and support small local businesses. We wanted to go beyond a static directory and create a tool that matches real customer needs with timely local offers.

### How is this different from Google Maps?

Google Maps is built around searching for places. Ping is built around structured local requests and offer matching. Users describe what they need, and the app ranks claimable offers based on fit, budget, distance, timing, ratings, and preferences.

### What is the intelligent feature?

OfferRank. It scores and ranks offers using category fit, budget, distance, rating, time availability, verification, and preferences. It also explains why each offer matched.

### How do you prevent absurd requests?

The app uses structured fields and semantic validation. Each need type has realistic budget rules, required fields, cooldowns, and validation checks.

## How do you prevent bot activity?

Users complete a mock verification step before matching, claim codes are generated, claims have cooldowns, reviews require redemption, and duplicate claims are blocked.

## What data structures did you use?

Arrays and typed objects store businesses, offers, users, requests, claims, reviews, saved items, and rankings. Personal rankings use ordered arrays, and the app uses binary insertion for efficient pairwise ranking.

## Why use binary insertion?

When placing a new business into a user's ranking, binary insertion reduces comparisons from linear time to logarithmic time. This improves UX because the user answers fewer comparison questions.

## What reports does the app generate?

Users can view savings, claims, redemptions, businesses supported, favorite categories, and rating patterns. Businesses can view offer views, claims, redemptions, conversion rate, ratings, and review tags.

---

# 23. Build Order for AI Agent

## Phase 1: Project Setup

1. Create React + TypeScript + Vite app.
2. Add routing.
3. Add global layout.
4. Add design tokens.
5. Add seed data.
6. Add storage service.

## Phase 2: Core Data and Services

1. Define TypeScript models.
2. Build request validation service.
3. Build offer matching service.
4. Build claim service.
5. Build review service.
6. Build report service.
7. Build ranking service.

## Phase 3: User App

1. Home page.
2. Create Ping page.
3. Verification modal.
4. Matches page.
5. Offer cards.
6. Claim flow.
7. Claims page.
8. Explore page.
9. Business profile page.
10. Saved page.

## Phase 4: Business App

1. Business dashboard.
2. Create offer page.
3. Manage offers page.
4. Redeem claim page.
5. Reviews page.
6. Analytics page.

## Phase 5: Reports and Rankings

1. User reports page.
2. Business analytics charts.
3. Pairwise comparison modal.
4. Rankings page.

## Phase 6: Polish

1. Empty states.
2. Error states.
3. Help panel.
4. Accessibility checks.
5. Demo reset.
6. Seed data improvements.
7. Final QA.

---

## 24. Final MVP Checklist

Before submission, the app must support:

- Create structured Ping request

- Validate request format and meaning
  - Complete mock verification
  - Match offers with OfferRank
  - Explain match reasons
  - Browse businesses by category
  - Sort by rating
  - Save businesses
  - Save offers
  - Claim offers
  - Generate claim code
  - Redeem claim as business
  - Leave verified review
  - Update business rating
  - Pairwise rank businesses
  - Show user report
  - Show business analytics
  - Persist data locally
  - Reset demo data
  - Run without internet
  - No console/runtime errors
  - Clear code organization
  - Clean visual design
  - Help/instructions available
  - Accessibility basics included
- 

## 25. Final Product Summary

Ping is a structured local offer-matching app for small businesses. Users create a specific request, the app validates it, ranks local offers using OfferRank, allows users to claim verified offers, and turns redemptions into trusted reviews, personal rankings, and reports. Businesses can publish offers, redeem claim codes, and analyze customer engagement.

The product is designed to clearly satisfy the FBLA Coding & Programming prompt while giving judges a strong demonstration of UX, logic, data structures, validation, reports, and intelligent matching.